
RAPPORT DE TP : LIVE CODING 6

Architecture Distribuée, Résilience et Observabilité

Réalisé par :
METOGHE OBIANG Simplicie Dariel

Encadré par :
M. AMAMOU Ahmed

1 Lab Sécurité — Mini Threat Modeling (STRIDE)

1.1 Objectif

Ce laboratoire applique la méthodologie **STRIDE** (Microsoft) pour identifier les menaces de sécurité sur l'architecture du Système de Gestion Documentaire Distribué (SGDD). Pour chaque type de menace, on analyse si elle est applicable à notre API et on propose une contre-mesure.

1.2 Rappel — Les 6 catégories STRIDE

Lettre	Menace	Question clé
S	Spoofing (usurpation)	Qui peut se faire passer pour un autre ?
T	Tampering (falsification)	Les données peuvent-elles être modifiées ?
R	Repudiation (non-répudiation)	Une action peut-elle être niée ?
I	Information Disclosure (fuite)	Des données sensibles fuient-elles ?
D	Denial of Service (DoS)	Le service peut-il être rendu indisponible ?
E	Elevation of Privilege (élévation)	Un utilisateur peut-il obtenir des droits supérieurs ?

TABLE 1 – Les 6 catégories de menace STRIDE.

1.3 Application STRIDE à l'API du SGDD

1.3.1 S — Spoofing (Usurpation d'identité)

- **Menace** : Un attaquant fabrique un faux token JWT pour se faire passer pour un utilisateur légitime.
- **Détection possible** : Signature JWT invalide, utilisation d'un token expiré.
- **Contre-mesure** :
 - Vérification systématique de la signature JWT par chaque service (clé publique)
 - Vérification de l'audience (aud) et de l'expiration (exp)
 - Utilisation de tokens courts (24h) avec refresh token

1.3.2 T — Tampering (Falsification des données)

- **Menace** : Un attaquant intercepte une requête HTTP et modifie le contenu d'un document en transit.
- **Détection possible** : Incohérence entre le document reçu et le document stocké.
- **Contre-mesure** :
 - TLS 1.3 obligatoire sur tous les endpoints
 - mTLS pour les communications inter-services
 - Signature des payloads critiques (HMAC) en complément de TLS

1.3.3 R — Repudiation (Non-répudiation)

- **Menace** : Un utilisateur supprime un document et nie l'avoir fait. Aucune trace ne permet de prouver le contraire.
- **Détection possible** : Absence de logs d'audit.
- **Contre-mesure** :

- Logs d’audit obligatoires pour les actions critiques (login, échec login, création, modification, suppression, accès admin)
- Logs immuables (envoi vers un système centralisé type ELK ou journalctl en mode append-only)
- Association de chaque action à un `user_id` et un `request_id`

1.3.4 I — Information Disclosure (Divulgarion d’informations)

- **Menace** : Les messages d’erreur HTTP révèlent des détails internes (stack trace, structure de la base de données, chemin du fichier).
- **Détection possible** : Réponse HTTP 500 contenant une stack trace Python complète.
- **Contre-mesure** :
 - Messages d’erreur génériques en production (ex : `{"error": "internal_server_error"}`)
 - Jamais de stack trace dans les réponses HTTP
 - Logs détaillés uniquement côté serveur (fichiers logs), jamais renvoyés au client

1.3.5 D — Denial of Service (Déni de service)

- **Menace** : Un attaquant envoie des milliers de requêtes POST /documents avec des fichiers volumineux pour saturer le stockage et le CPU.
- **Détection possible** : Pic soudain de requêtes, CPU à 100%, erreurs 503.
- **Contre-mesure** :
 - Rate limiting : 10 req/min/IP sur /login, 30 req/min/user sur /search
 - Taille maximale des documents (ex : 10 Mo)
 - Pagination forcée (max 100 résultats)
 - Timeout serveur à 30 secondes
 - Circuit breaker côté client (après N échecs)

1.3.6 E — Elevation of Privilege (Élévation de privilèges)

- **Menace** : Un utilisateur modifie son token JWT pour changer son rôle de reader à admin.
- **Détection possible** : Signature JWT invalide (si l’attaquant ne possède pas la clé privée).
- **Contre-mesure** :
 - Signature JWT avec clé privée (RS256) – l’utilisateur ne peut pas modifier les claims sans invalider la signature
 - Vérification des rôles à chaque endpoint (AuthZ)
 - Moindre privilège : un rôle reader ne peut pas accéder aux endpoints d’administration
 - Token scope : différencier token utilisateur et token service (audience différente)

1.4 Matrice STRIDE résumée pour l'API SGDD

Menace STRIDE	Exemple concret SGDD	Contre-mesure	Priorité
Spoofing	Faux token JWT	Vérification signature + audience	Critique
Tampering	Modification document en transit	TLS + mTLS	Critique
Repudiation	Suppression d'un document niée	Logs d'audit immuables	Élevée
Info Disclosure	Stack trace dans réponse HTTP	Messages d'erreur génériques	Critique
DoS	Requêtes massives POST /documents	Rate limiting + taille max	Élevée
Elevation	Changement de rôle dans JWT	Signature JWT + AuthZ endpoint	Critique

TABLE 2 – Récapitulatif STRIDE pour l'API du SGDD.

1.5 Checklist de sécurité minimale (API Baseline)

Cette checklist a été vérifiée pour chaque endpoint de l'API :

- TLS/HTTPS activé sur tous les endpoints (y compris internes)
- Authentification requise sur tous les endpoints sauf GET /health
- Autorisation vérifiée pour chaque opération (pas seulement l'authentification)
- Validation d'entrée sur tous les champs (type, longueur, format, whitelist)
- Rate limiting configuré (auth : 5 req/min, lecture : 30 req/min, écriture : 10 req/min)
- Pagination forcée sur les endpoints de liste (max 100 par page)
- Messages d'erreur génériques (pas de stack trace en production)
- Pas de secrets dans le code source (variables d'environnement ou Vault)
- Rotation des secrets planifiée (tous les 90 jours minimum)
- Audit logs sur : login, échec login, création, suppression, accès admin
- X-Request-ID propagé à tous les services

1.6 Étude d'incident — Token compromis

1.6.1 Scénario

Un développeur a accidentellement commité un fichier .env contenant le token du service Document dans un dépôt Git public. Le token donne un accès complet en lecture/écriture à tous les documents. Le commit a été poussé il y a 3 heures.

1.6.2 Plan de réponse (dans l'ordre)

Phase	Action	Justification
Containment	Révoquer immédiatement le token compromis (invalidation côté Auth Service)	Bloquer tout accès malveillant en cours – chaque minute compte
Rotation	Générer un nouveau token et déployer sur le Document Service	Rétablir le fonctionnement normal avec des credentials sains
Audit	Analyser les logs d'accès des 3 dernières heures (requêtes avec ce token, IPs, actions)	Déterminer si le token a été exploité par un tiers
Évaluation	Identifier les données touchées, notifier les utilisateurs concernés	Obligation légale (RGPD) si fuite de données personnelles
Nettoyage	Supprimer le fichier .env du dépôt (git filter-branch ou BFG)	Le secret ne doit plus être accessible, même dans l'historique
Prévention	Ajouter .env au .gitignore, outil de détection de secrets dans le CI (pre-commit, scanning)	Empêcher que l'incident se reproduise
Post-mortem	Rédiger un rapport d'incident (timeline, cause racine, impact, actions correctives)	Culture d'amélioration continue (blameless post-mortem)

TABLE 3 – Plan de réponse à incident – Token compromis.

Piège classique – « J'ai supprimé le commit, c'est bon »

Si le dépôt est public, des bots scannent GitHub en continu et peuvent avoir copié le secret en quelques secondes. La **révocation immédiate du token** est toujours la première action, avant même de nettoyer le dépôt.

1.7 Conclusion du Lab Sécurité

L'application de la méthode STRIDE à notre API a révélé plusieurs points critiques :

1. L'authentification par token JWT doit être systématiquement vérifiée (signature, audience, expiration)
2. Les logs d'audit sont indispensables pour la non-répudiation
3. Les messages d'erreur ne doivent jamais exposer de détails internes
4. Le rate limiting est la première ligne de défense contre le DoS
5. La rotation des secrets et la réponse à incident doivent être planifiées avant la mise en production

À retenir pour l'examen / le projet :

- STRIDE est un cadre de réflexion, pas un outil magique
- Chaque menace identifiée doit avoir une contre-mesure technique
- La sécurité se pense dès l'architecture, pas en ajout tardif
- Zero Trust : ne jamais faire confiance au réseau interne